

Order Delivery Time Prediction

Objectives

The objective of this assignment is to build a regression model that predicts the delivery time for orders placed through Porter. The model will use various features such as the items ordered, the restaurant location, the order protocol, and the availability of delivery partners.

The key goals are:

- Predict the delivery time for an order based on multiple input features
- Improve delivery time predictions to optimiae operational efficiency
- Understand the key factors influencing delivery time to enhance the model's accuracy

Data Pipeline

The data pipeline for this assignment will involve the following steps:

1. **Data Loading**
2. **Data Preprocessing and Feature Engineering**
3. **Exploratory Data Analysis**
4. **Model Building**
5. **Model Inference**

Data Understanding

The dataset contains information on orders placed through Porter, with the following columns:

Field	Description
market_id	Integer ID representing the market where the restaurant is located.
created_at	Timestamp when the order was placed.
actual_delivery_time	Timestamp when the order was delivered.
store_primary_category	Category of the restaurant (e.g., fast food, dine-in).
order_protocol	Integer representing how the order was placed (e.g., via Porter, call to restaurant, etc.).
total_items	Total number of items in the order.
subtotal	Final price of the order.
num_distinct_items	Number of distinct items in the order.
min_item_price	Price of the cheapest item in the order.
max_item_price	Price of the most expensive item in the order.
total_onshift_dashers	Number of delivery partners on duty when the order was placed.
total_busy_dashers	Number of delivery partners already occupied with other orders.
total_outstanding_orders	Number of orders pending fulfillment at the time of the order.
distance	Total distance from the restaurant to the customer.

Importing Necessary Libraries

```
# Import essential libraries for data manipulation and analysis

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

1. Loading the data

Load 'porter_data_1.csv' as a DataFrame

```
# Importing the file porter_data_1.csv
from google.colab import files
uploaded = files.upload()
```

```
import pandas as pd
```

```
df = pd.read_csv('porter_data_1.csv')
print(df.head())
```

```
Choose Files porter_data_1.csv
• porter_data_1.csv(text/csv) - 15667264 bytes, last modified: 5/1/2025 - 100% done
Saving porter_data_1.csv to porter_data_1.csv
  market_id  created_at  actual_delivery_time \
0         1.0  2015-02-06 22:24:17  2015-02-06 23:11:17
1         2.0  2015-02-10 21:49:25  2015-02-10 22:33:25
2         2.0  2015-02-16 00:11:35  2015-02-16 01:06:35
3         1.0  2015-02-12 03:36:46  2015-02-12 04:35:46
4         1.0  2015-01-27 02:12:36  2015-01-27 02:58:36

  store_primary_category  order_protocol  total_items  subtotal \
0                      4                1.0           4       3441
1                      46                2.0           1       1900
2                      36                3.0           4       4771
3                      38                1.0           1       1525
4                      38                1.0           2       3620

  num_distinct_items  min_item_price  max_item_price  total_onshift_dashers \
0                   4              557           1239                33.0
1                   1             1400           1400                 1.0
2                   3              820           1604                 8.0
3                   1             1525           1525                 5.0
4                   2             1425           2195                 5.0

  total_busy_dashers  total_outstanding_orders  distance
0                14.0                   21.0      34.44
1                 2.0                   2.0      27.60
2                 6.0                   18.0      11.56
3                 6.0                   8.0      31.80
4                 5.0                   7.0       8.20
```

✓ 2. Data Preprocessing and Feature Engineering [15 marks]

✓ 2.1 Fixing the Datatypes [5 marks]

The current timestamps are in object format and need conversion to datetime format for easier handling and intended functionality

✓ 2.1.1 [2 marks]

Convert date and time fields to appropriate data type

```
# Convert 'created_at' and 'actual_delivery_time' columns to datetime format
```

```
df['created_at'] = pd.to_datetime(df['created_at'])
df['actual_delivery_time'] = pd.to_datetime(df['actual_delivery_time'])
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175777 entries, 0 to 175776
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   market_id                            175777 non-null float64
1   created_at                           175777 non-null datetime64[ns]
2   actual_delivery_time                 175777 non-null datetime64[ns]
3   store_primary_category              175777 non-null int64
4   order_protocol                      175777 non-null float64
5   total_items                         175777 non-null int64
6   subtotal                            175777 non-null int64
7   num_distinct_items                 175777 non-null int64
8   min_item_price                     175777 non-null int64
9   max_item_price                     175777 non-null int64
10  total_onshift_dashers               175777 non-null float64
11  total_busy_dashers                 175777 non-null float64
12  total_outstanding_orders           175777 non-null float64
13  distance                           175777 non-null float64
dtypes: datetime64[ns](2), float64(6), int64(6)
memory usage: 18.8 MB
```

✓ 2.1.2 [3 marks]

Convert categorical fields to appropriate data type

```
# Convert categorical features to category type
```

```
categorical_features = ['market_id', 'store_primary_category', 'order_protocol']
for feature in categorical_features:
    df[feature] = df[feature].astype('category')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175777 entries, 0 to 175776
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   market_id                            175777 non-null  category
1   created_at                           175777 non-null  datetime64[ns]
2   actual_delivery_time                 175777 non-null  datetime64[ns]
3   store_primary_category               175777 non-null  category
4   order_protocol                       175777 non-null  category
5   total_items                          175777 non-null  int64
6   subtotal                            175777 non-null  int64
7   num_distinct_items                  175777 non-null  int64
8   min_item_price                      175777 non-null  int64
9   max_item_price                      175777 non-null  int64
10  total_onshift_dashers                175777 non-null  float64
11  total_busy_dashers                   175777 non-null  float64
12  total_outstanding_orders             175777 non-null  float64
13  distance                             175777 non-null  float64
dtypes: category(3), datetime64[ns](2), float64(4), int64(5)
memory usage: 15.3 MB
```

✓ 2.2 Feature Engineering [5 marks]

Calculate the time taken to execute the delivery as well as extract the hour and day at which the order was placed

✓ 2.2.1 [2 marks]

Calculate the time taken using the features `actual_delivery_time` and `created_at`

```
# Calculate time taken in minutes
df['time_taken'] = (df['actual_delivery_time'] - df['created_at']).dt.total_seconds() / 60
```

✓ 2.2.2 [3 marks]

Extract the hour at which the order was placed and which day of the week it was. Drop the unnecessary columns.

```
# Extract the hour and day of week from the 'created_at' timestamp
df['order_hour'] = df['created_at'].dt.hour
df['order_day_of_week'] = df['created_at'].dt.dayofweek

# Create a categorical feature 'isWeekend'
df['isWeekend'] = np.where(df['order_day_of_week'].isin([5, 6]), 1, 0)
```

```
# Drop unnecessary columns
df = df.drop(['created_at', 'actual_delivery_time'], axis=1)
```

✓ 2.3 Creating training and validation sets [5 marks]

✓ 2.3.1 [2 marks]

Define target and input features

```
# Define target variable (y) and features (X)

target_variable = 'time_taken'
feature_columns = df.columns.drop(target_variable).tolist()

y = df[target_variable]
X = df[feature_columns]
```

✓ 2.3.2 [3 marks]

Split the data into training and test sets

```
# Split data into training and testing sets

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

✓ 3. Exploratory Data Analysis on Training Data [20 marks]

1. Analyzing the correlation between variables to identify patterns and relationships
2. Identifying and addressing outliers to ensure the integrity of the analysis
3. Exploring the relationships between variables and examining the distribution of the data for better insights

✓ 3.1 Feature Distributions [7 marks]

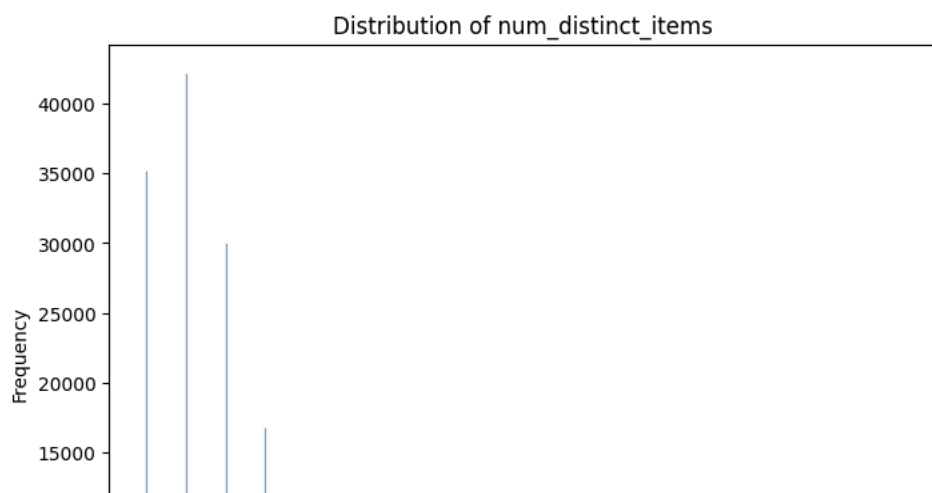
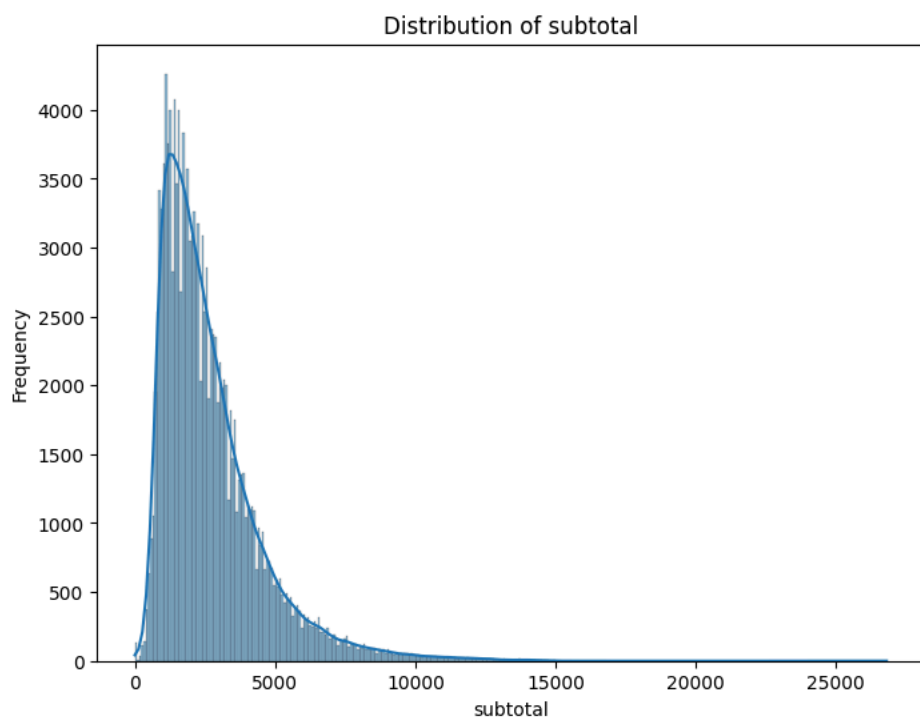
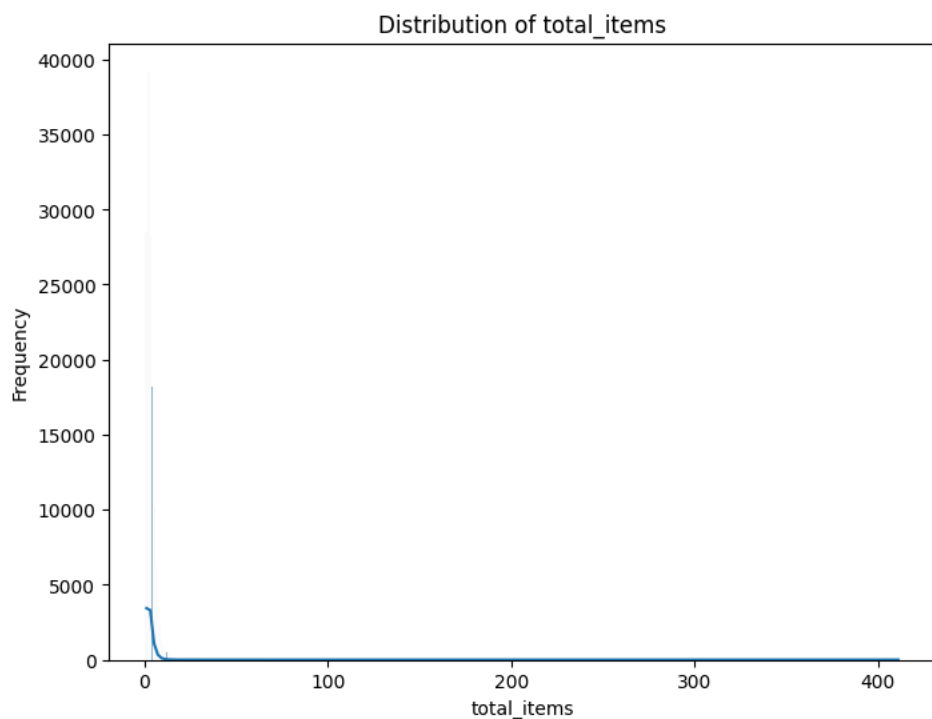
```
# Define numerical and categorical columns for easy EDA and data manipulation
numerical_cols = X_train.select_dtypes(include=np.number).columns.tolist()
categorical_cols = X_train.select_dtypes(include=['object', 'category']).columns.tolist()
```

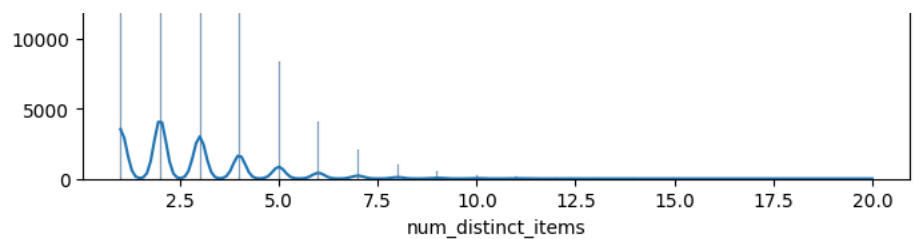
✓ 3.1.1 [3 marks]

Plot distributions for numerical columns in the training set to understand their spread and any skewness

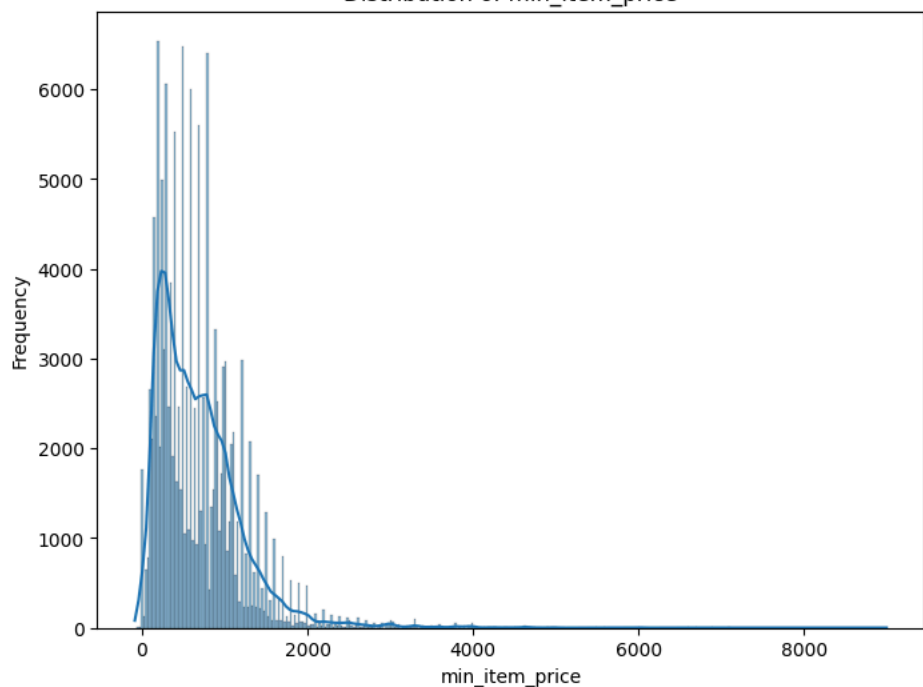
```
# Plot distributions for all numerical columns
import matplotlib.pyplot as plt
import seaborn as sns

for col in numerical_cols:
    plt.figure(figsize=(8, 6)) # Adjust figure size as needed
    sns.histplot(X_train[col], kde=True) # kde=True adds a kernel density estimate
    plt.title(f'Distribution of {col}')
    plt.xlabel(col)
    plt.ylabel('Frequency')
    plt.show()
```

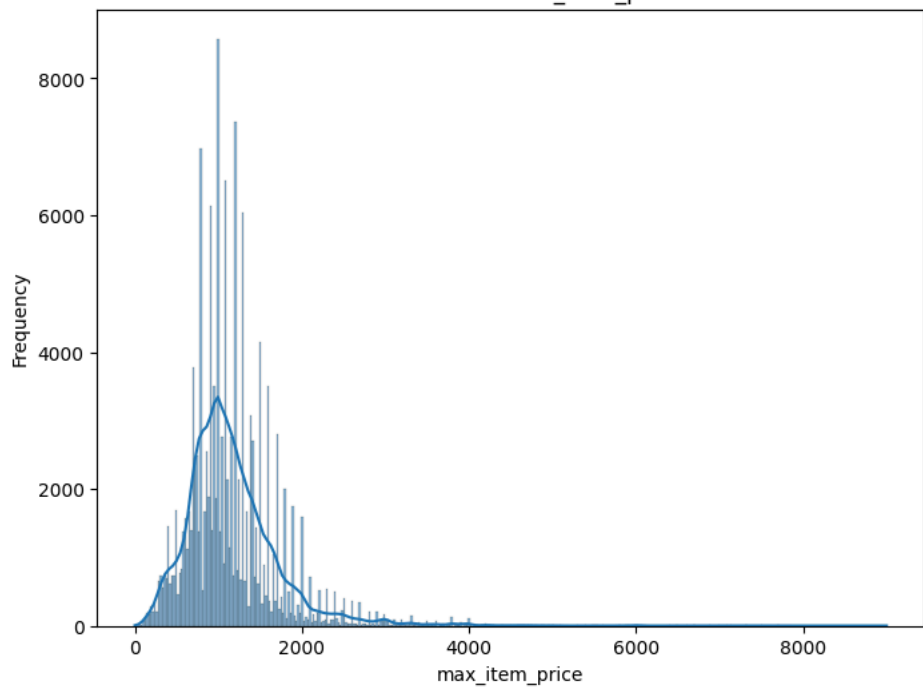




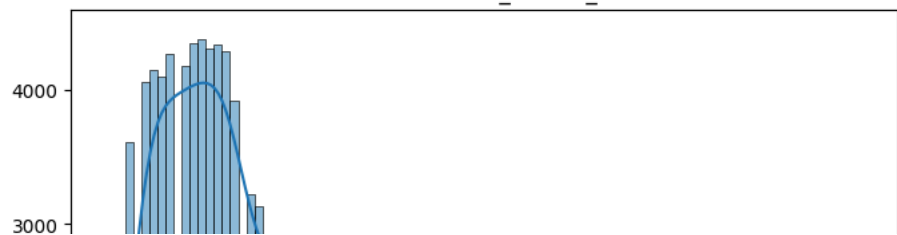
Distribution of min_item_price

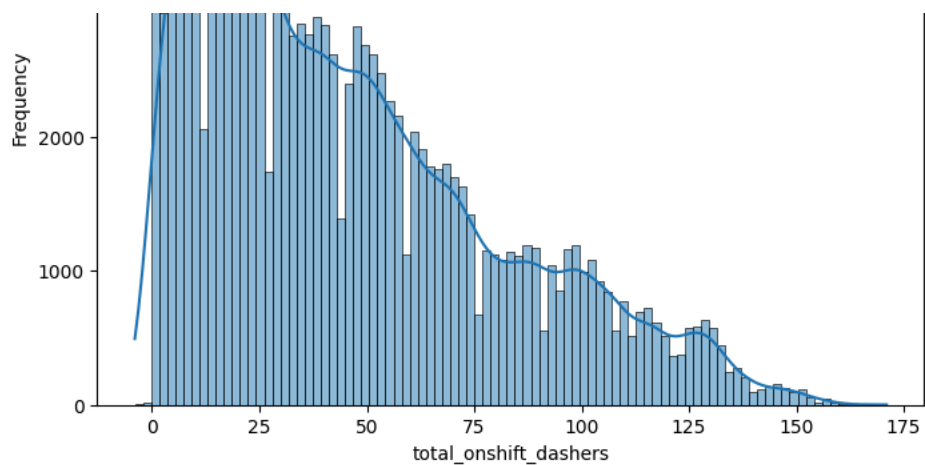


Distribution of max_item_price

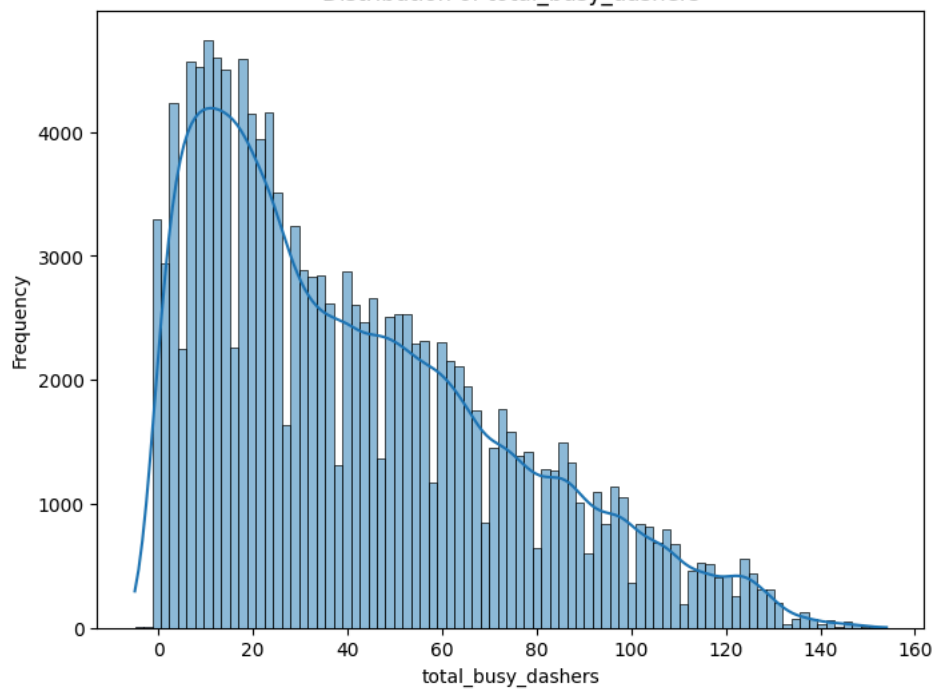


Distribution of total_onshift_dashers

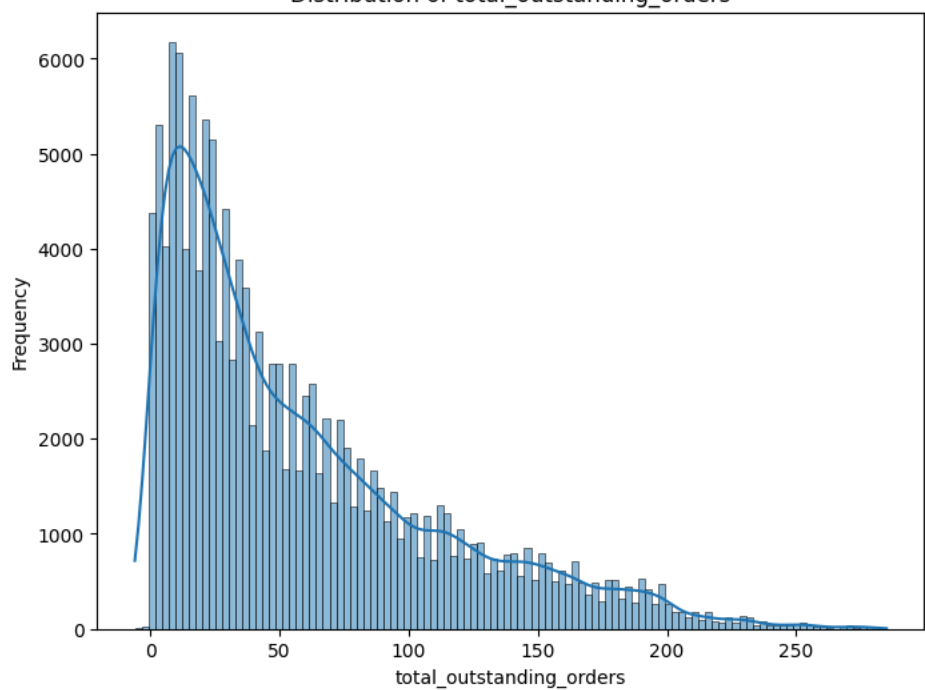




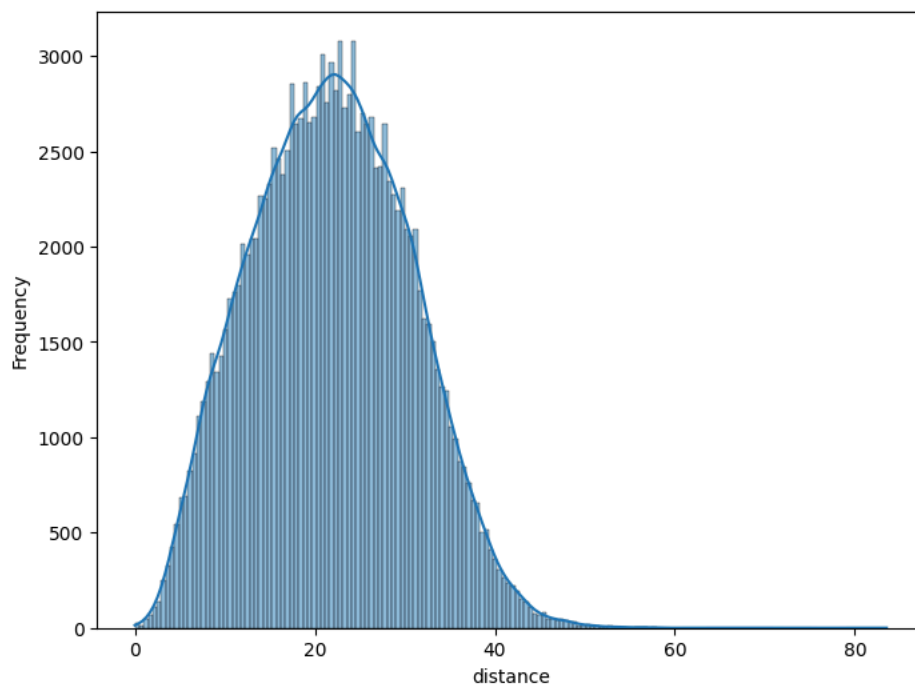
Distribution of total_busy_dashers



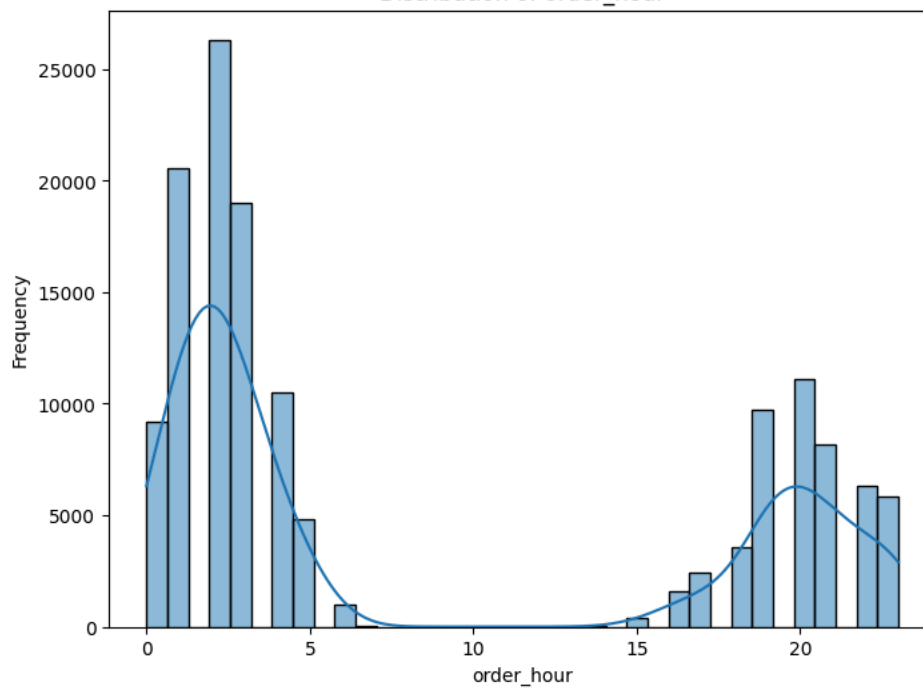
Distribution of total_outstanding_orders



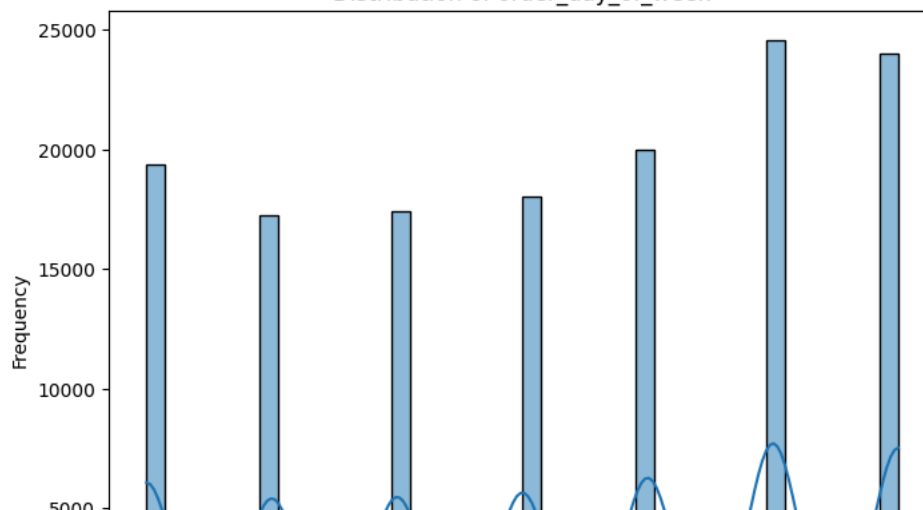
Distribution of distance

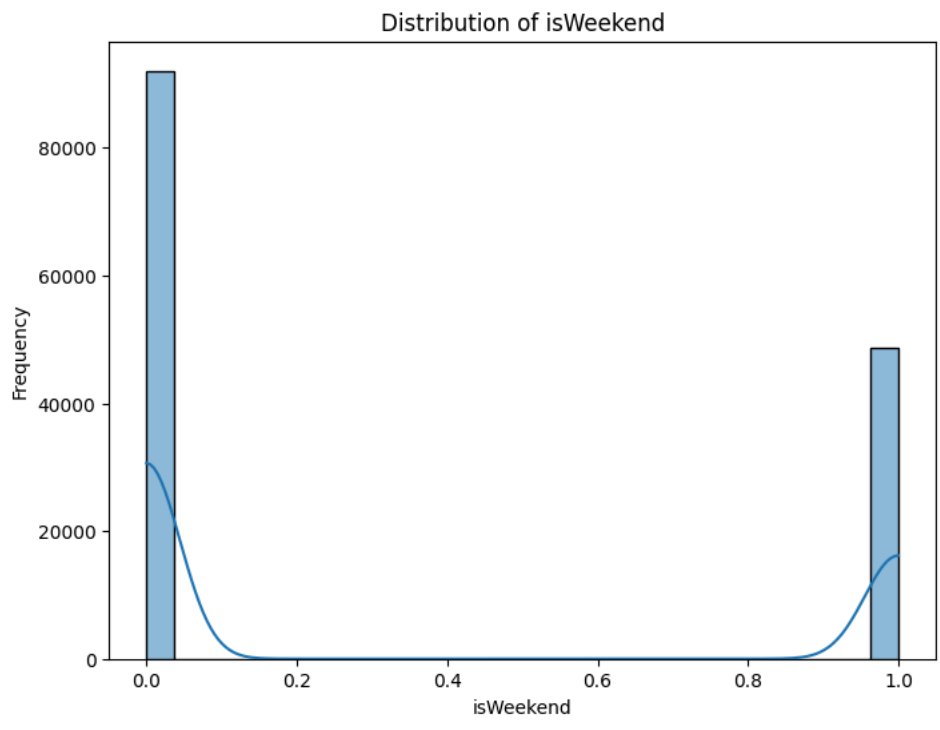
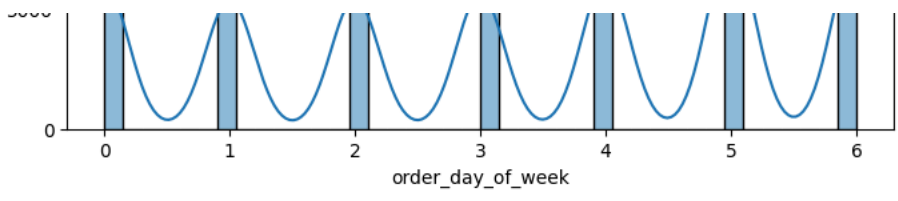


Distribution of order_hour



Distribution of order_day_of_week

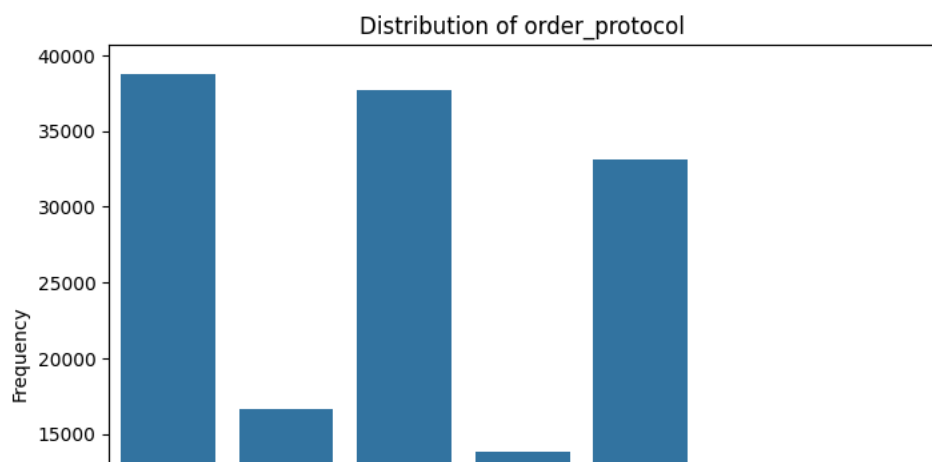
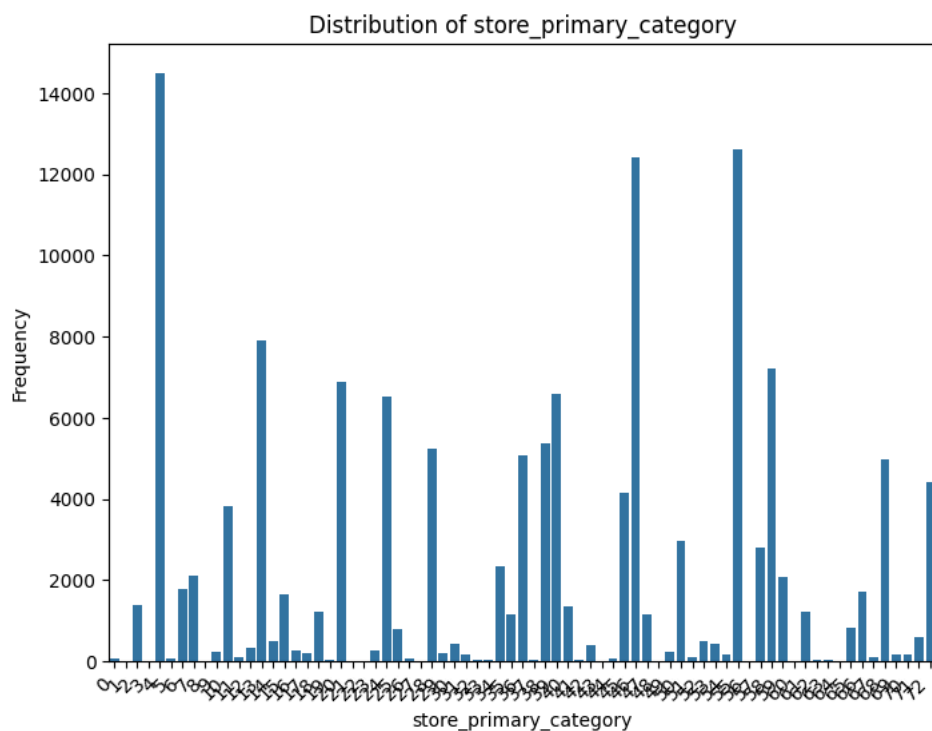
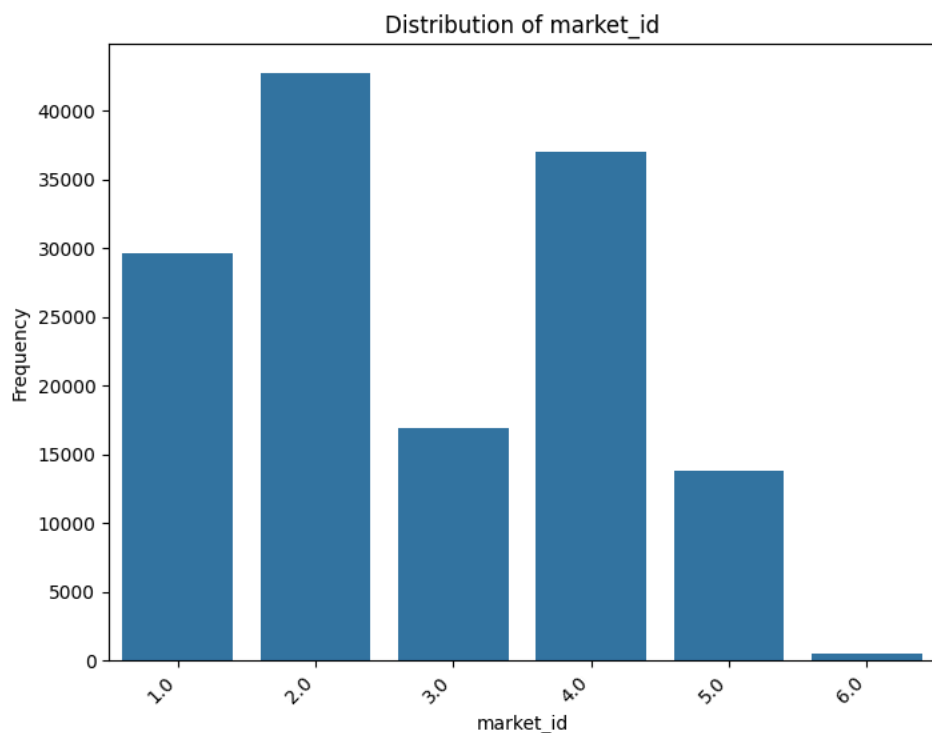


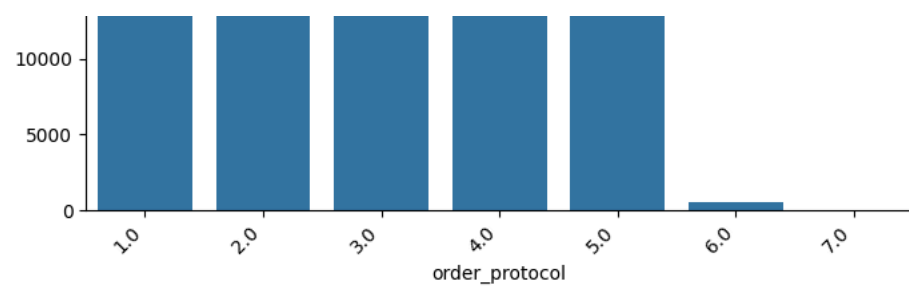


✓ **3.1.2** [2 marks]

Check the distribution of categorical features

```
# Distribution of categorical columns
for col in categorical_cols:
    plt.figure(figsize=(8, 6)) # Adjust figure size as needed
    sns.countplot(x=X_train[col]) # kde=True adds a kernel density estimate
    plt.title(f'Distribution of {col}')
    plt.xlabel(col)
    plt.ylabel('Frequency')
    plt.xticks(rotation=45, ha='right') # Rotate x-axis labels for better readability
    plt.show()
```





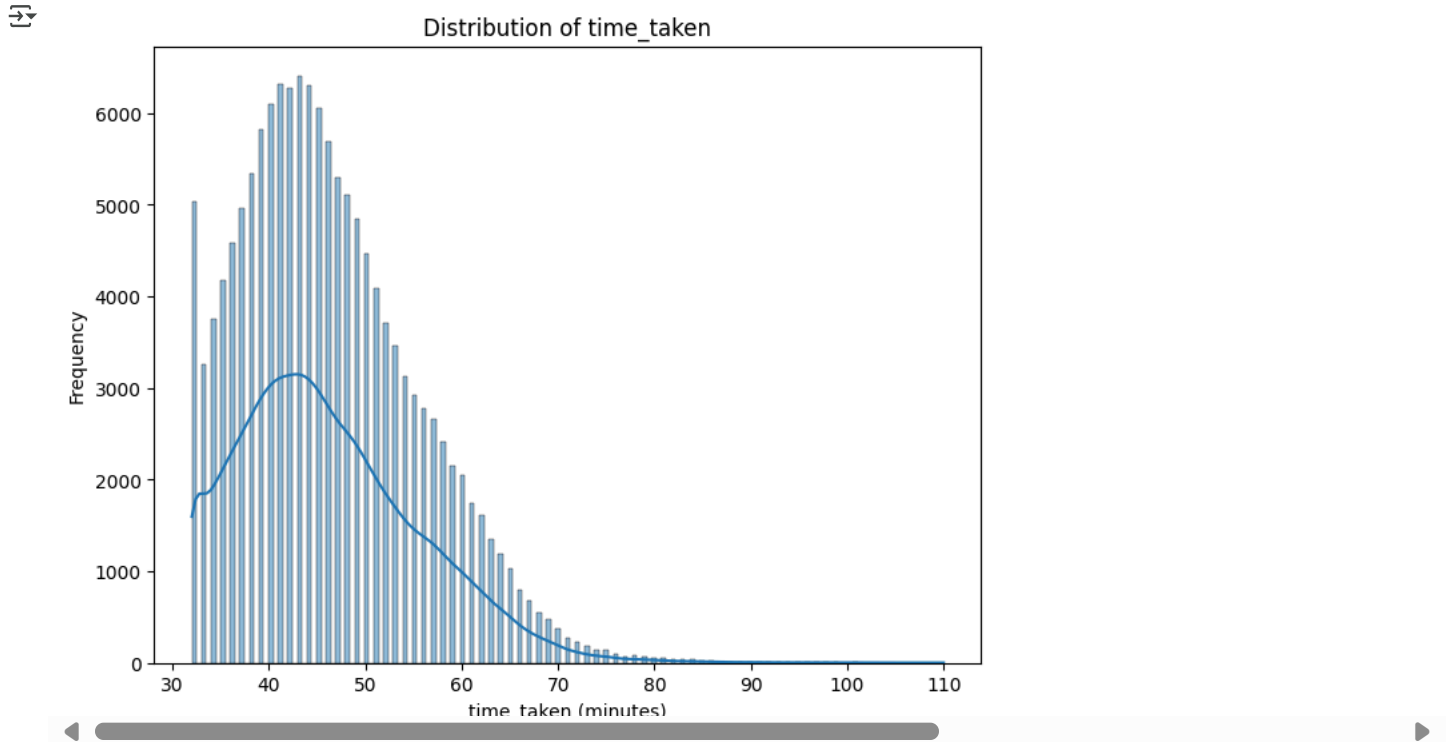
✓ 3.1.3 [2 mark]

Visualise the distribution of the target variable to understand its spread and any skewness

```
# Distribution of time_taken

import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8, 6)) # Adjust figure size as needed
sns.histplot(y_train, kde=True) # kde=True adds a kernel density estimate
plt.title('Distribution of time_taken')
plt.xlabel('time_taken (minutes)')
plt.ylabel('Frequency')
plt.show()
```



✓ 3.2 Relationships Between Features [3 marks]

✓ 3.2.1 [3 marks]

Scatter plots for important numerical and categorical features to observe how they relate to `time_taken`

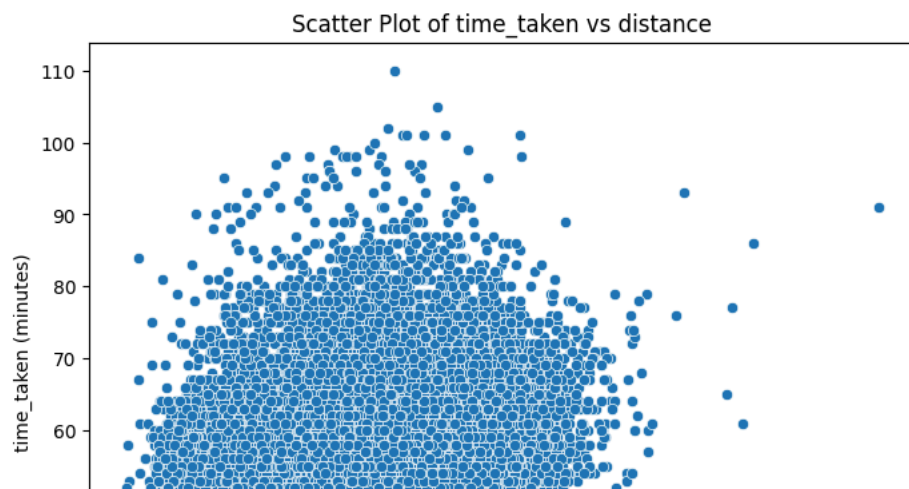
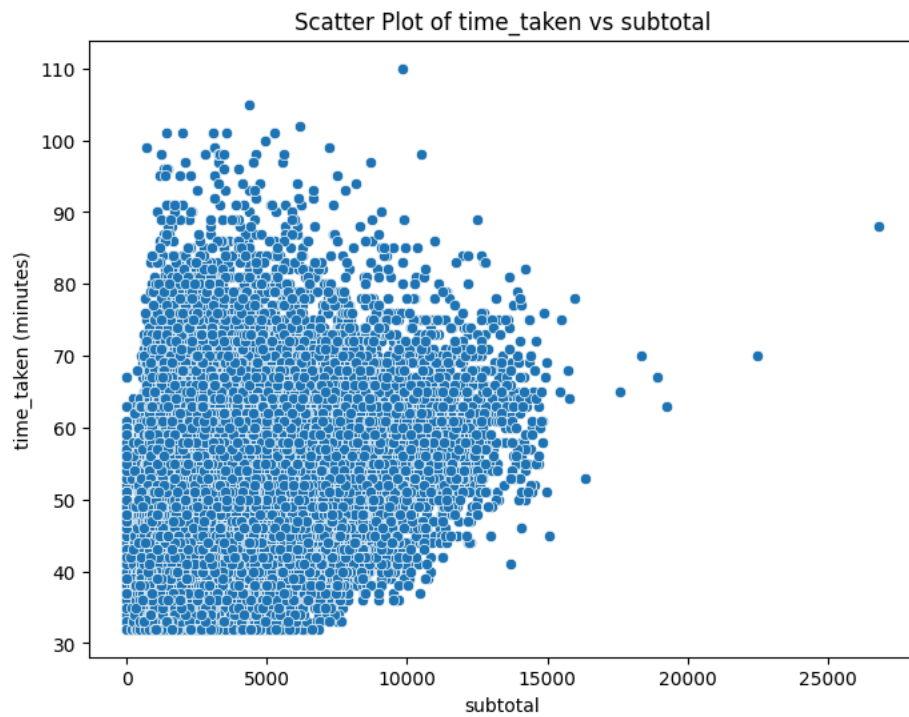
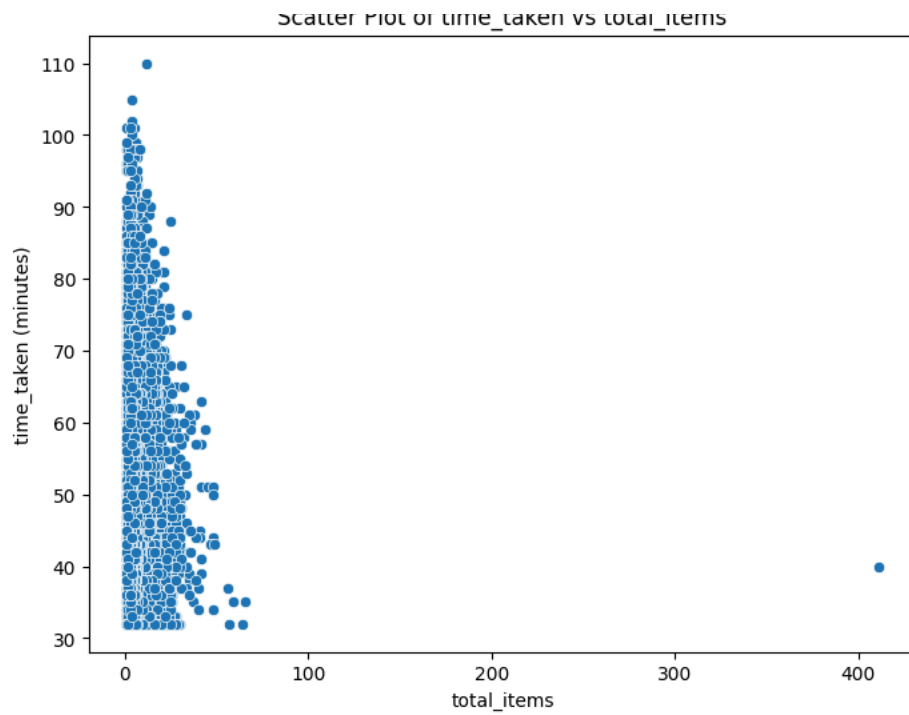
```
# Scatter plot to visualise the relationship between time_taken and other features

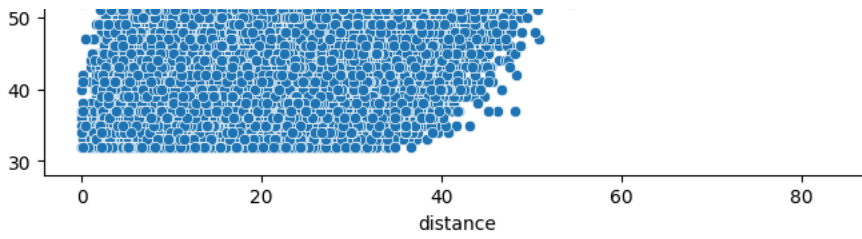
import matplotlib.pyplot as plt
import seaborn as sns

# Select the features to plot against 'time_taken'
features_to_plot = ['total_items', 'subtotal', 'distance', 'total_onshift_dashers', 'total_busy_dashers']

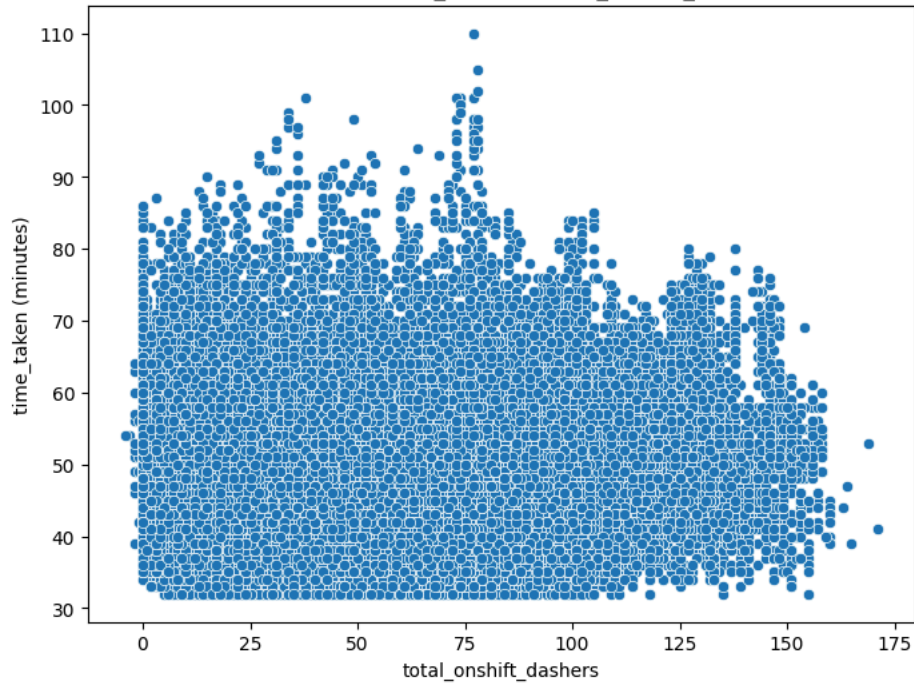
# Create scatter plots for each selected feature
for feature in features_to_plot:
    plt.figure(figsize=(8, 6)) # Adjust figure size as needed
    sns.scatterplot(x=X_train[feature], y=y_train)
    plt.title(f'Scatter Plot of time_taken vs {feature}')
    plt.xlabel(feature)
    plt.ylabel('time_taken (minutes)')
    plt.show()
```

[4]

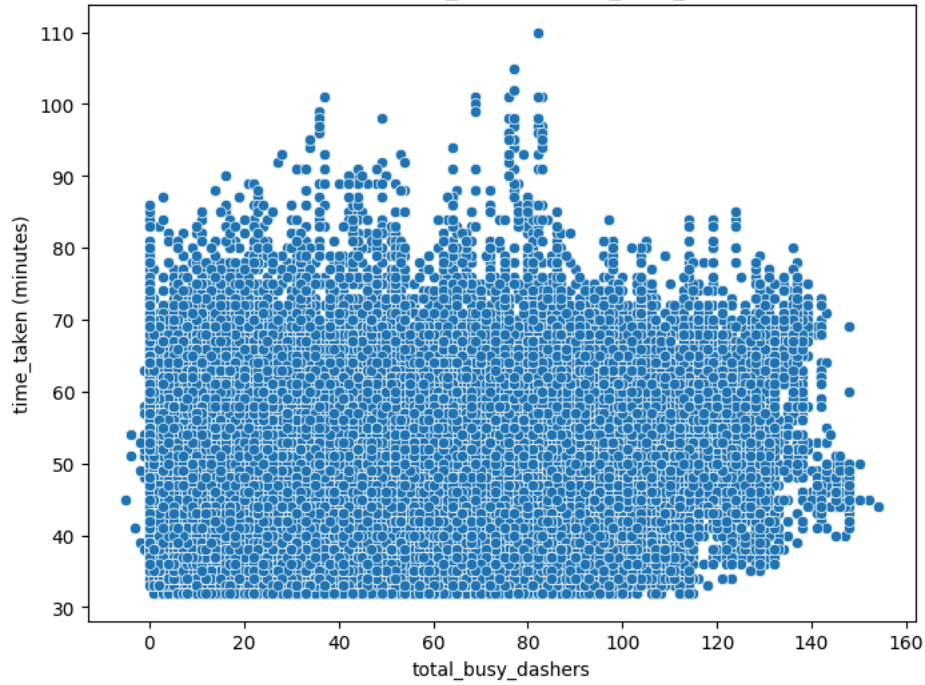




Scatter Plot of time_taken vs total_onshift_dashers



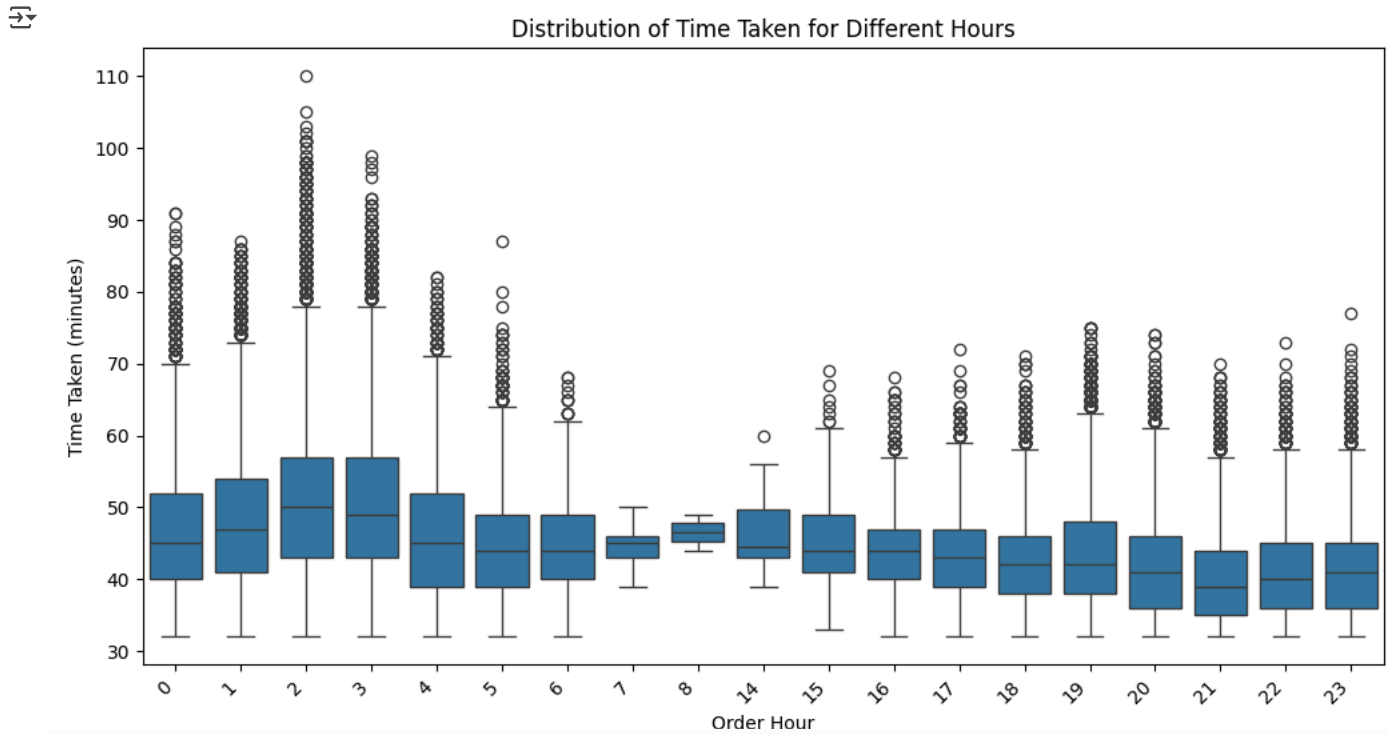
Scatter Plot of time_taken vs total_busy_dashers



```
# Show the distribution of time_taken for different hours
```

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(12, 6)) # Adjust figure size as needed
sns.boxplot(x='order_hour', y='time_taken', data=df)
plt.title('Distribution of Time Taken for Different Hours')
plt.xlabel('Order Hour')
plt.ylabel('Time Taken (minutes)')
plt.xticks(rotation=45, ha='right') # Rotate x-axis labels for better readability
plt.show()
```



✓ 3.3 Correlation Analysis [5 marks]

Check correlations between numerical features to identify which variables are strongly related to `time_taken`

✓ 3.3.1 [3 marks]

Plot a heatmap to display correlations

```
# Plot the heatmap of the correlation matrix
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# Calculate the correlation matrix
correlation_matrix = X_train.corr()
```

```
# Plot the heatmap
plt.figure(figsize=(12, 10)) # Adjust figure size as needed
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix Heatmap')
plt.show()
```